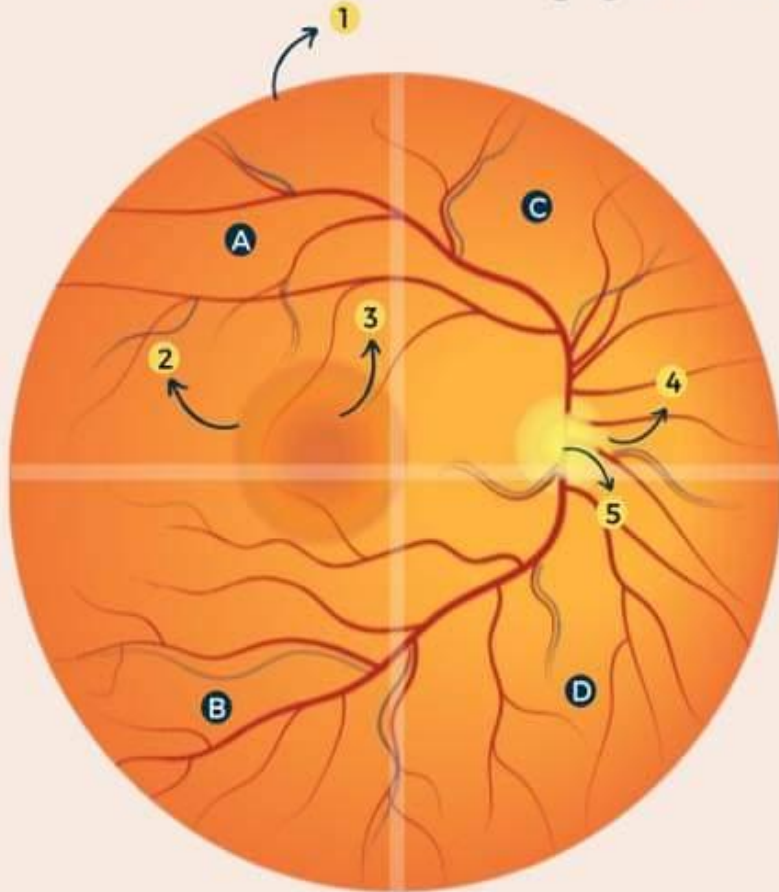




Interpretabilidad en modelos de Visión por Computadora

Caso de Estudio: Fondo de ojos

Imagen de fondo de ojos



1. RETINA
2. MÁCULA LÚTEA
3. FOVEA CENTRAL
4. DISCO ÓPTICO (PÁPILA)
5. EXCAVACIÓN DEL DISCO/PÁPILA

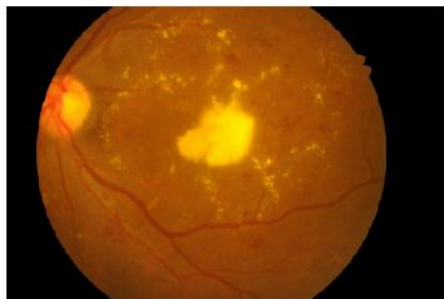
A. ARTERIAS Y VENAS TEMPORALES SUPERIORES

B. ARTERIAS Y VENAS TEMPORALES INFERIORES

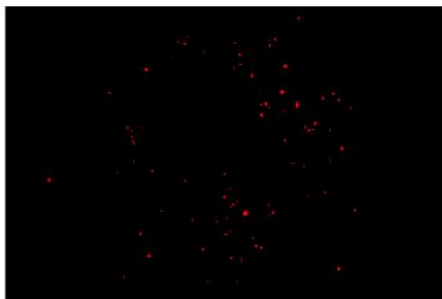
C. ARTERIAS Y VENAS NASALES SUPERIORES

D. ARTERIAS Y VENAS NASALES INFERIORES

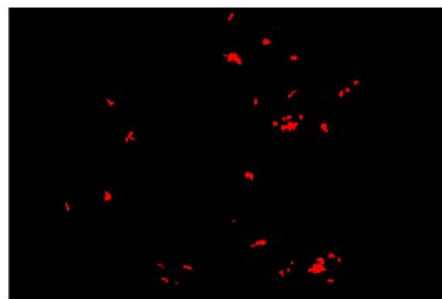
Retinopatía diabética



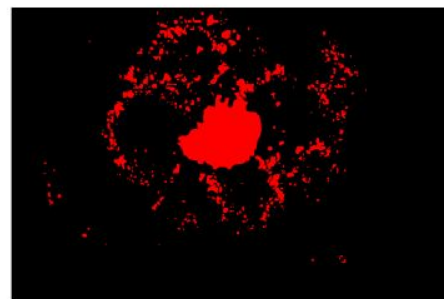
Fundus image



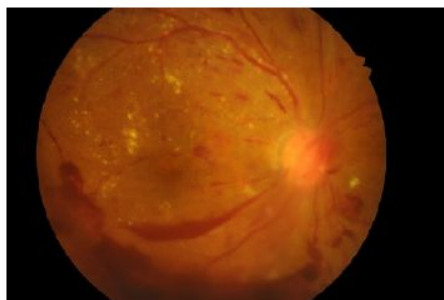
Microaneurysms



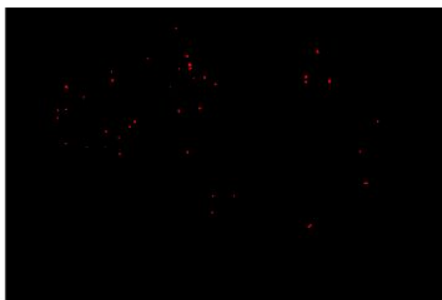
Haemorrhages



Hard exudates



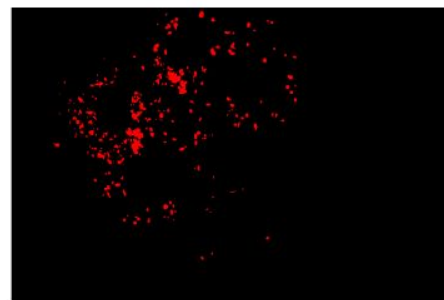
Fundus image



Microaneurysms



Haemorrhages



Hard exudates

Máscaras de segmentación de lesiones en el dataset Idris ([ejemplos](#))

EyePacs - Intensidad de enfermedad

- EyePacs: 5 clases, 90000 muestras
- 0 = No, 1 = Poco, 2 = Media, 3 = Severa, 4 = Degenerativa

- Original vs Kaggle
- Top F-score: 0.98 (2020)



(a) Proliferative DR



(b) No DR

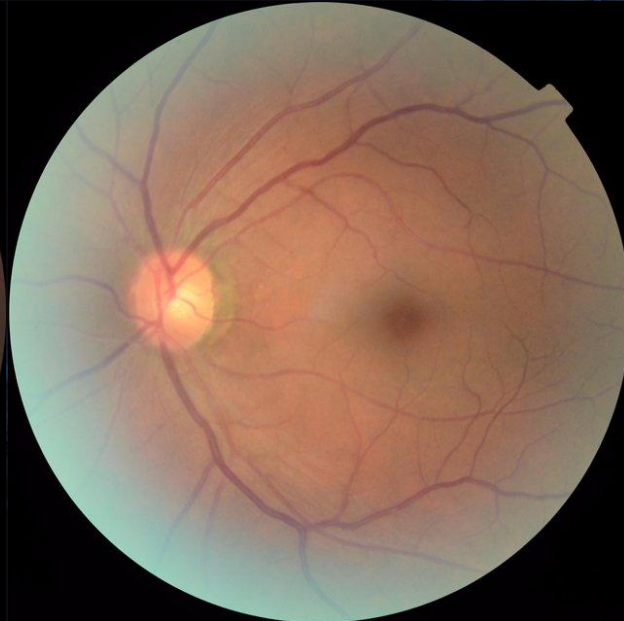
EyePacs - Gradability o Aceptabilidad

• Top acc: 0.87

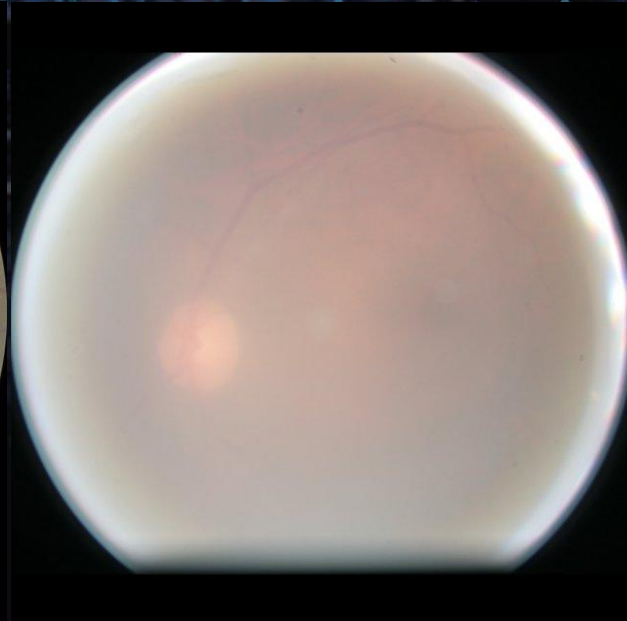
- 25% de las imágenes de EyePacs
 - Agregar una a 6th clase “No aceptable” al modelo
 - O crear un modelo de aceptabilidad



0: Acceptable

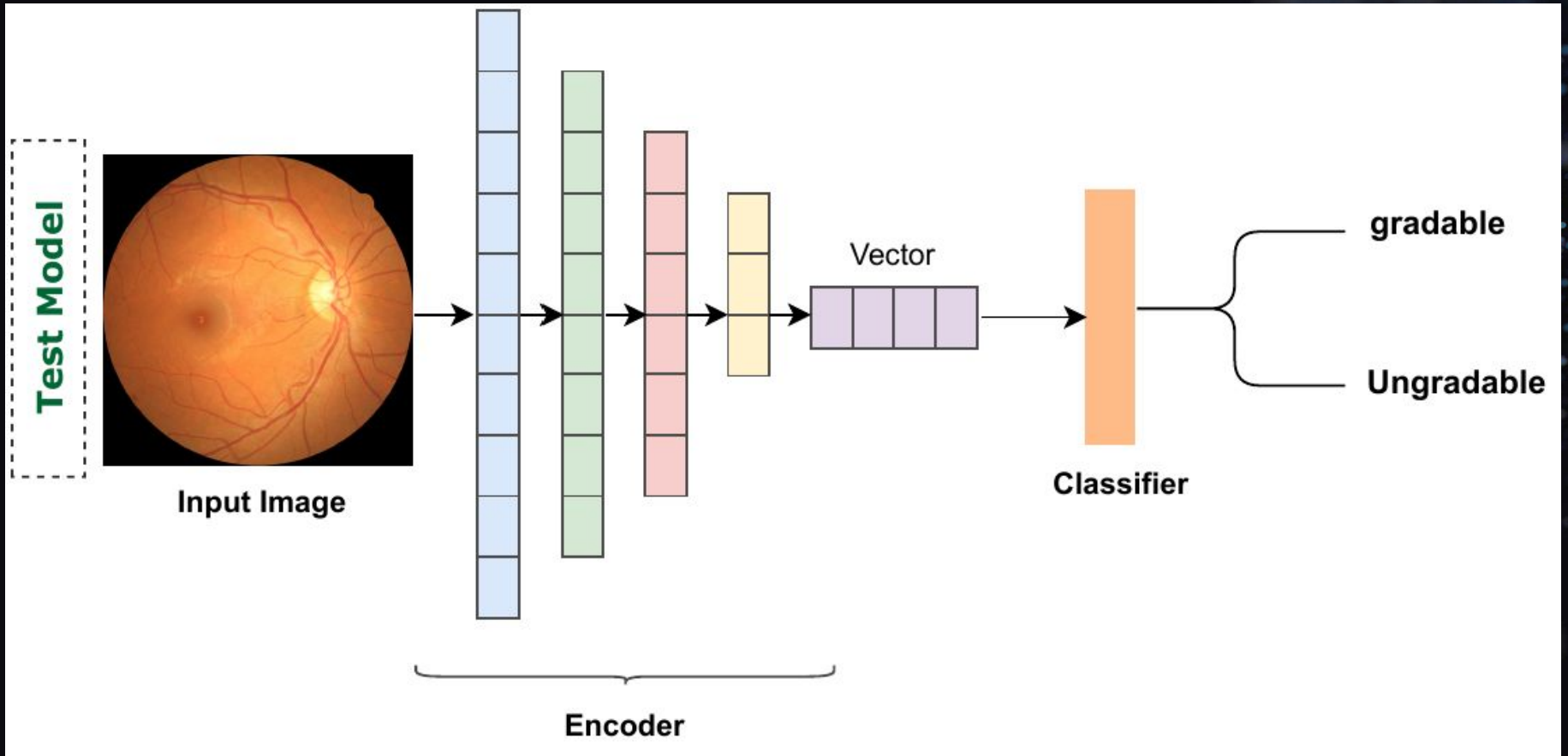


1: Mediocre

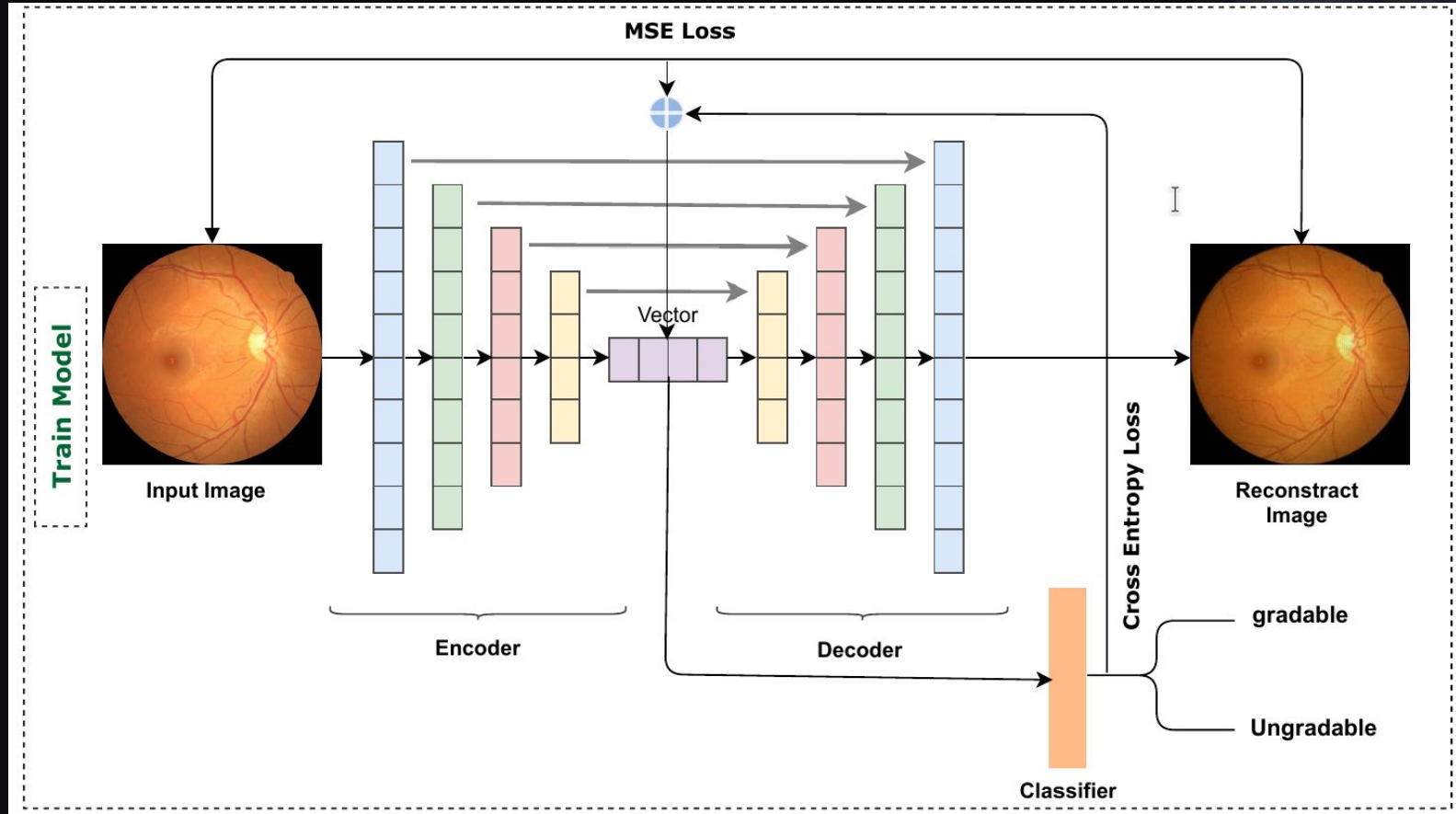


2: No aceptable

Modelo Simple para Aceptabilidad en Eyepacs



Modelo Avanzado para Aceptabilidad en Eyepacs



Resultados

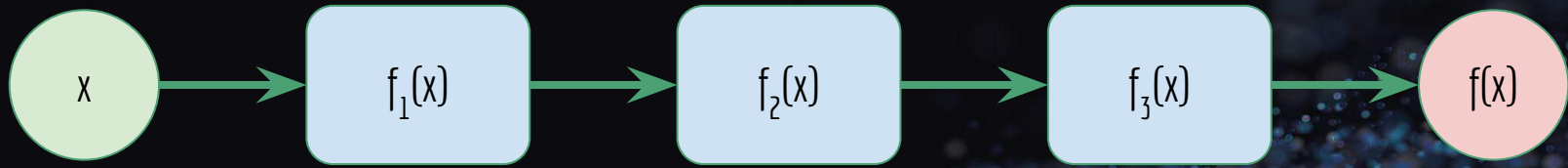
[1] Saif et al,
Efficient Fundus Image
Gradeability
Approach Based on
Deep
Reconstruction
Classification Network

Modelo	F. Error	#Classes	Accuracy
Simple [1]	-	2	0.81
Avanzado [1]	SSIM	2	0.815
Avanzado [1]	MS-SSIM	2	0.86
Avanzado [1]	MAE	2	0.85
Avanzado [1]	MSE	2	0.875
Simple	-	2	0.87
Avanzado+DA	MSE	3	0.85
Simple+DA	-	3	0.86

Interpretabilidad

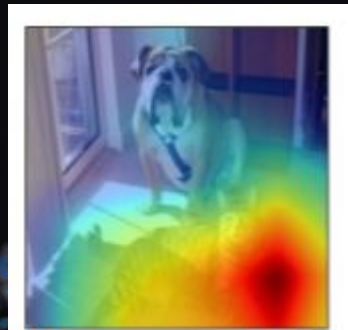
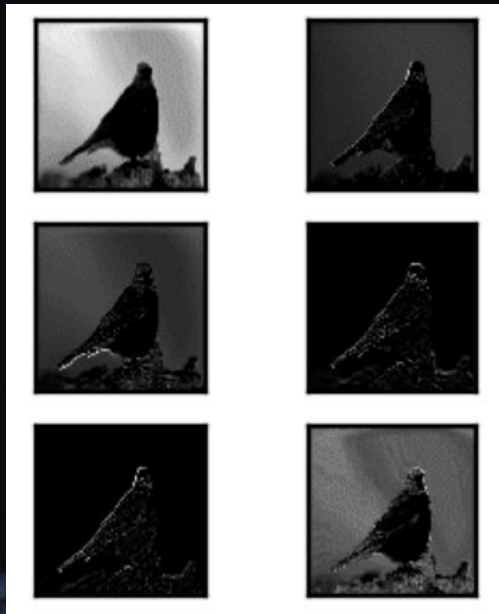
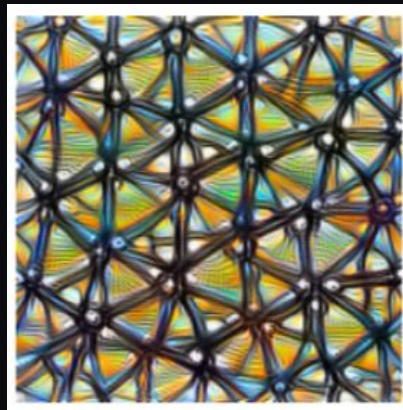
The background of the slide is a dark, almost black, space filled with numerous small, bright blue and white particles. These particles are arranged in several distinct, curved, and overlapping trails that sweep across the frame from the bottom left towards the top right. The trails have a soft, ethereal quality, with some particles appearing more sharply than others, creating a sense of depth and movement. The overall effect is reminiscent of a cosmic dust cloud or a visualization of data points in a high-dimensional space.

Vista relevante de una red neuronal para Interpretabilidad

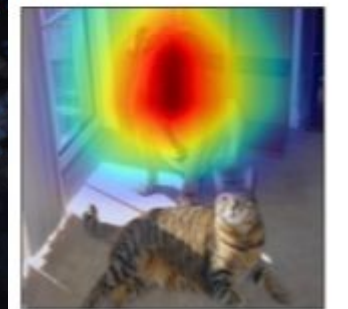


- Valores intermedios o **features**
 - $f_1(x), f_2(x), f_3(x)$
- Posibles derivadas/gradientes
 - $\partial f(x) / \partial x \rightarrow$ cómo afecta la entrada a la salida
 - $\partial f_i(x) / \partial x \rightarrow$ cómo afecta la entrada al feature i
 - $\partial f(x) / \partial f_i(x) \rightarrow$ cómo afecta el feature i a la salida
 - $\partial f_j(x) / \partial f_i(x) \rightarrow$ cómo afecta el feature i al feature j

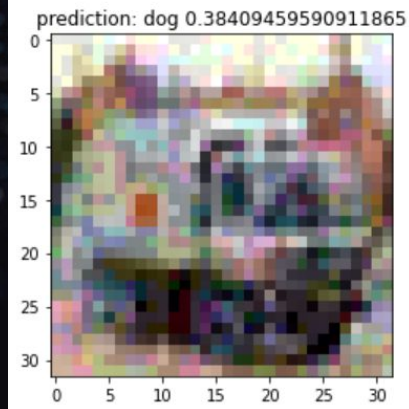
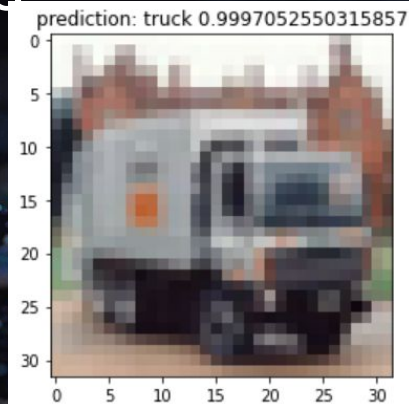
Algunos enfoques para interpretar modelos



(f) ResNet Grad-CAM 'Cat'



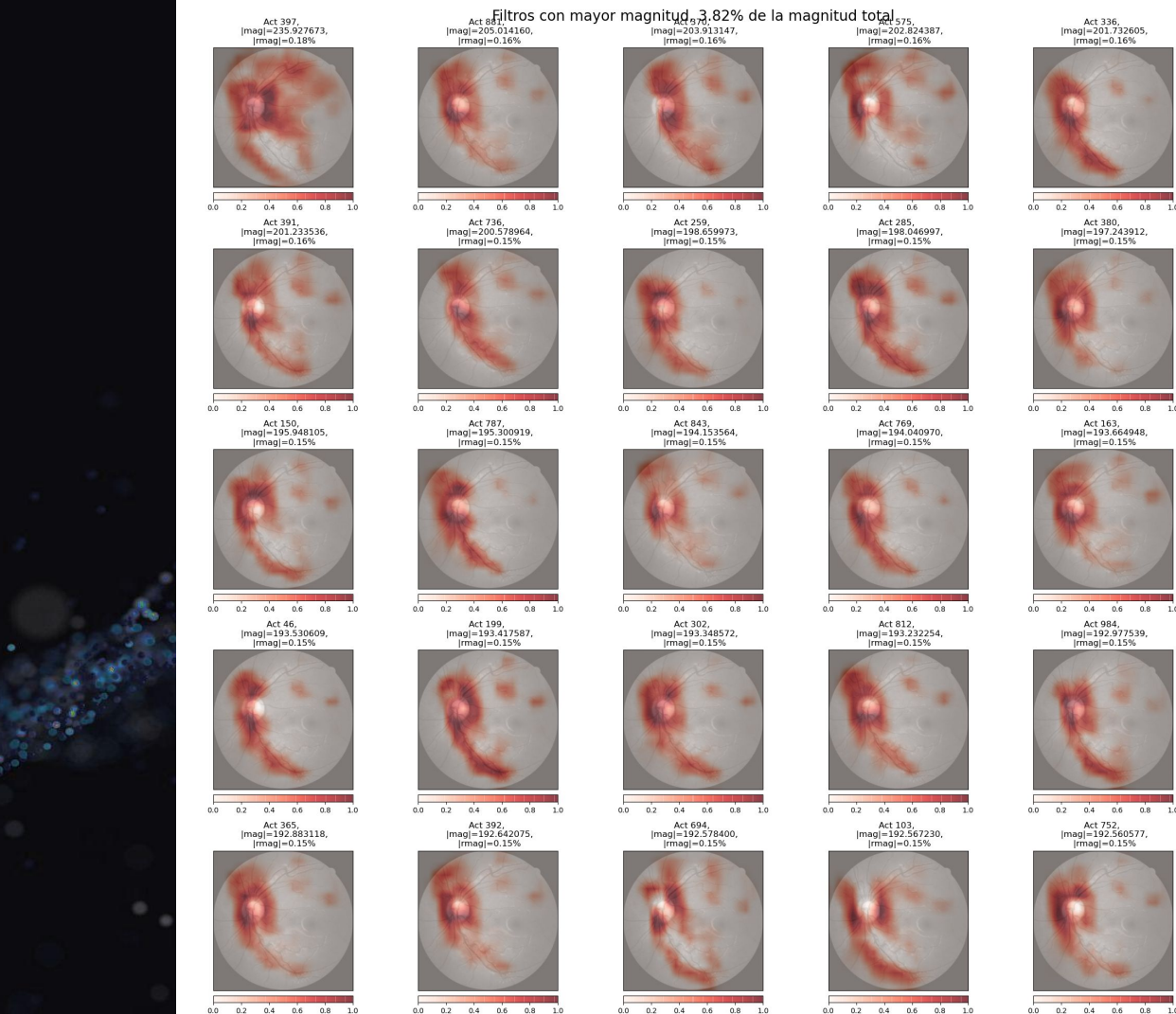
(l) ResNet Grad-CAM 'Dog'



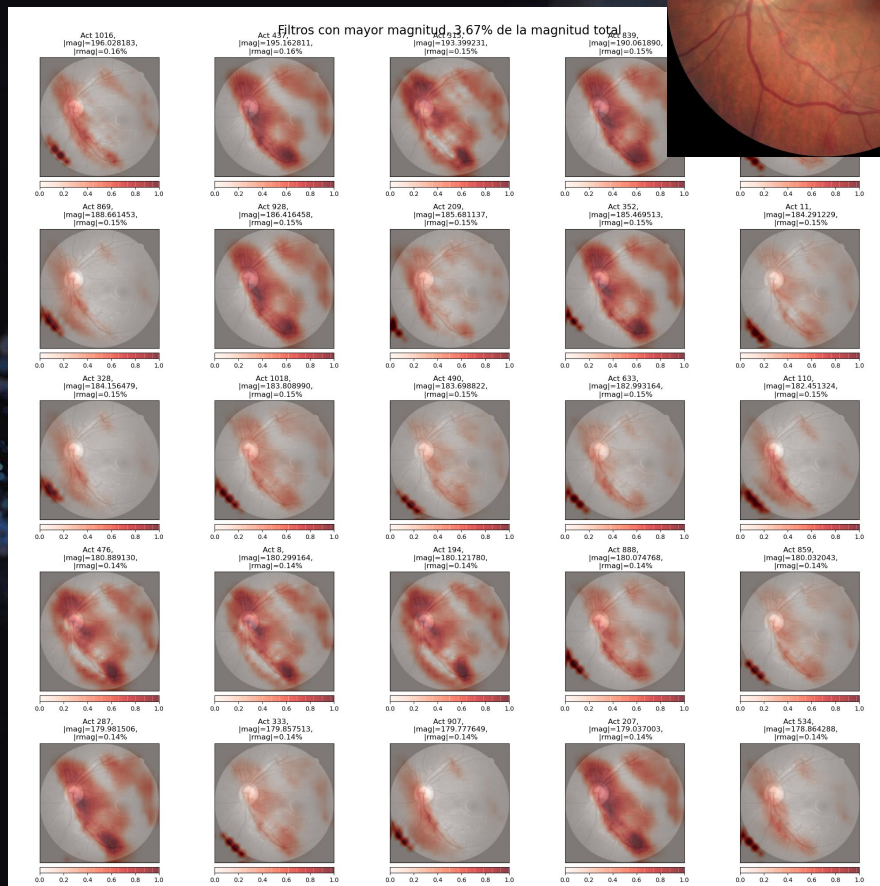
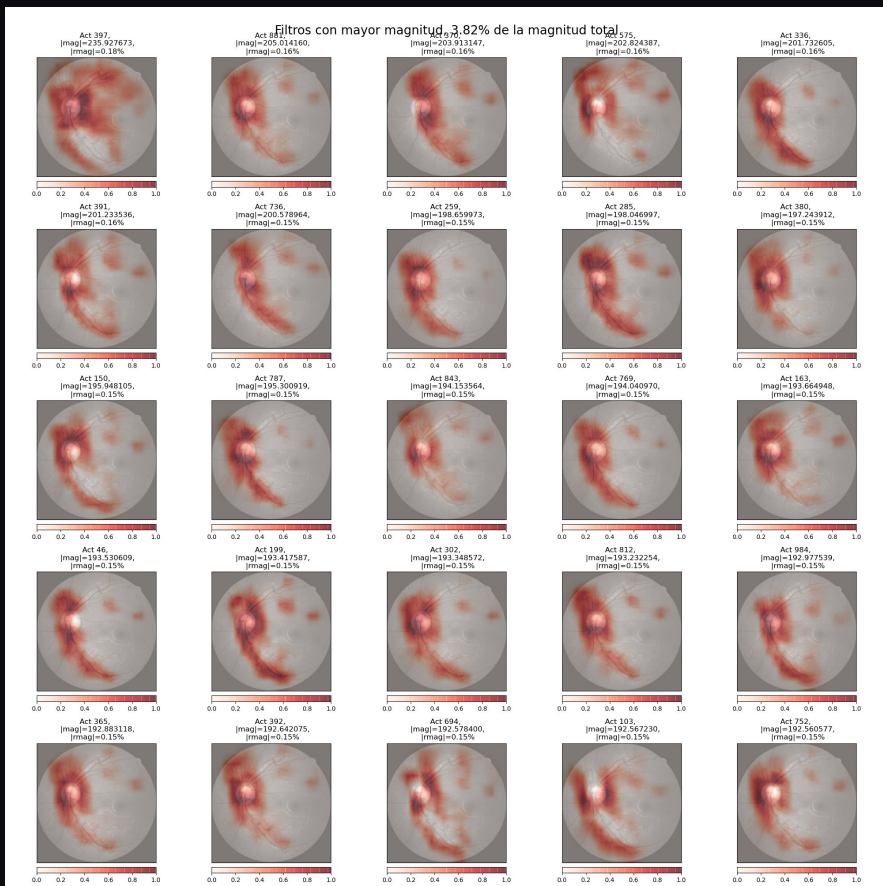
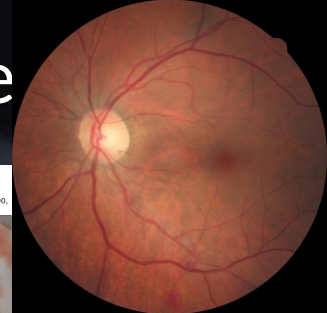
- Generativos
- Activaciones
- Atribución
- Adversarios

Activaciones

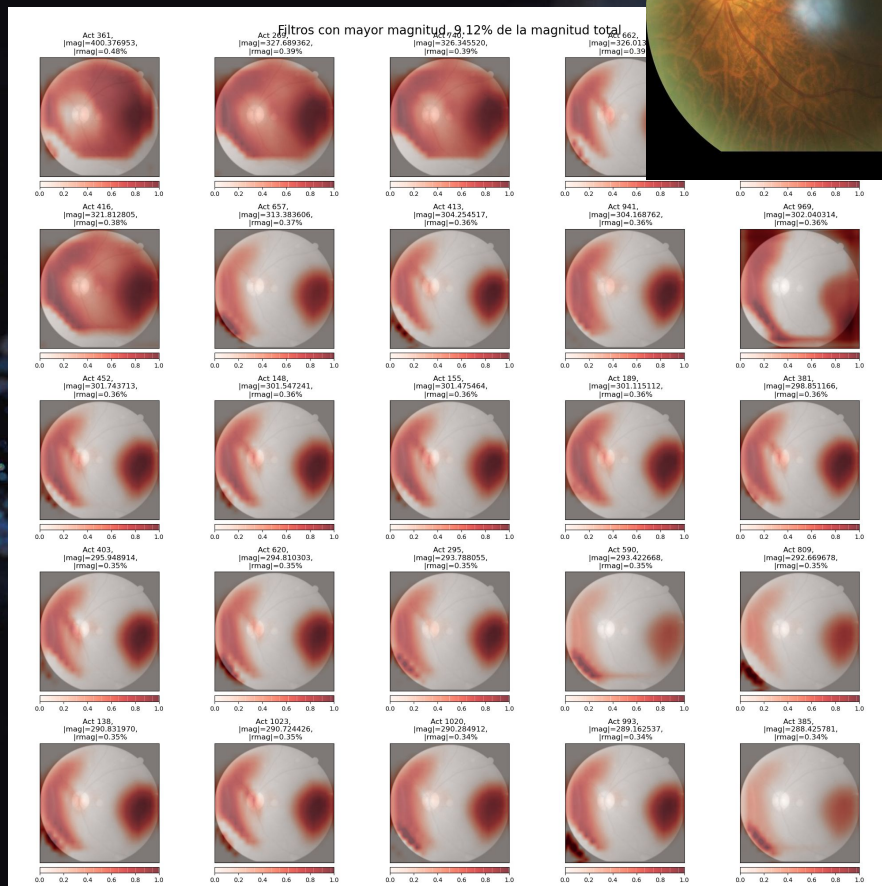
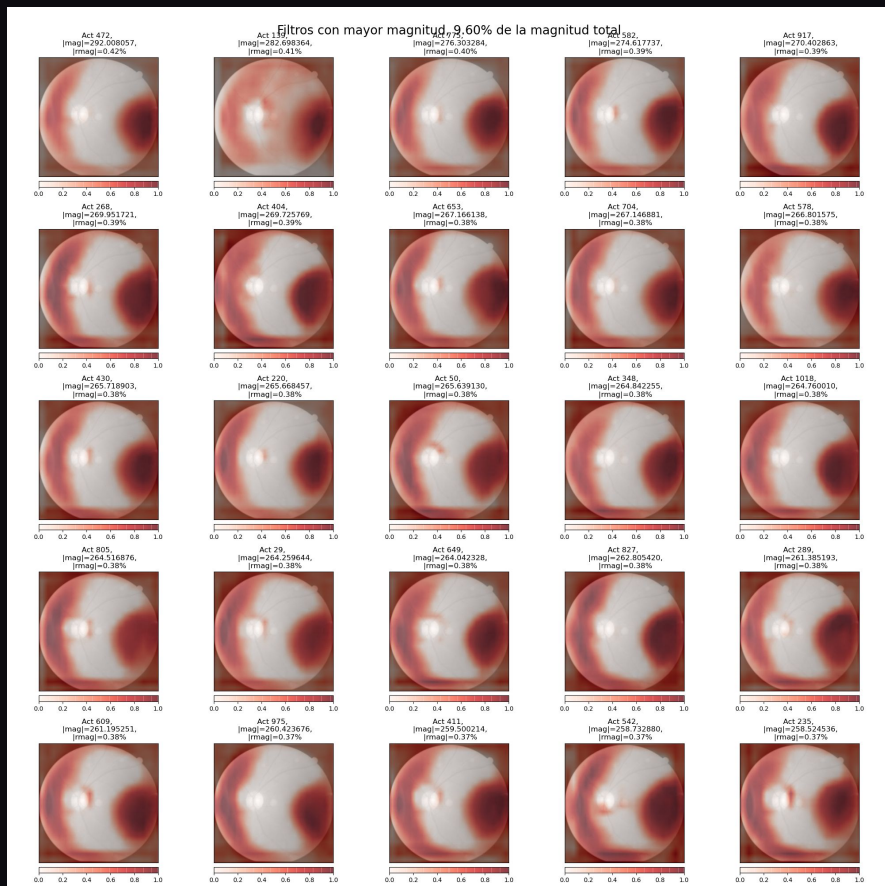
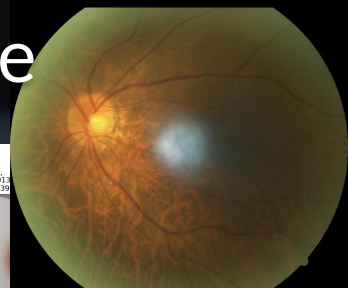
- Simplemente visualizar $f_i(x)$
- Demasiados features
 - Ordenar por magnitud
 - Puntaje de diversidad



Avanzado (izq) vs Simple (der): Clase aceptable

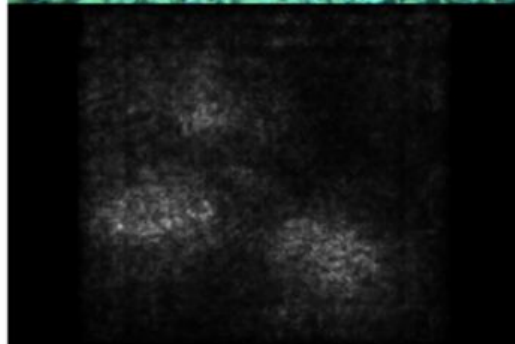
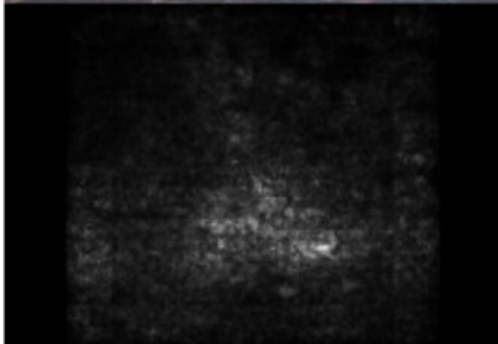
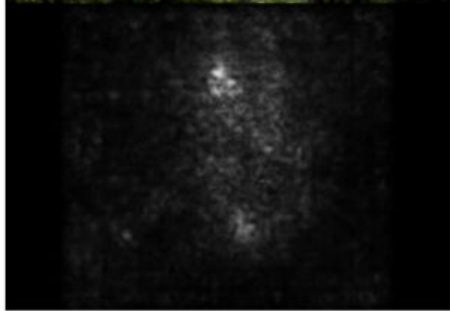


Avanzado (izq) vs Simple (der): C. No aceptable



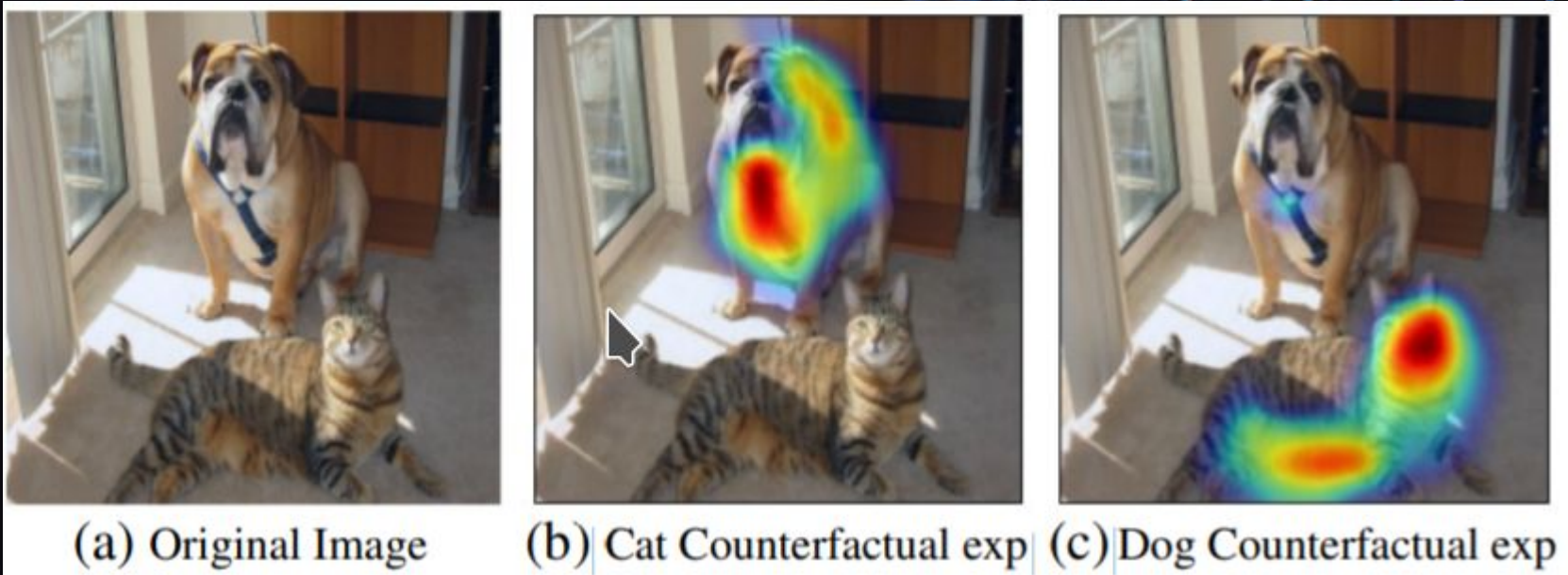
Gradiente

- Visualizar $\delta f(x)/\delta x$
 - Mismo tamaño que entrada
- O $\delta \text{puntaje_clase}(x)/\delta x$
 - Ver $\delta f(x)/\delta x$ como mapa de calor



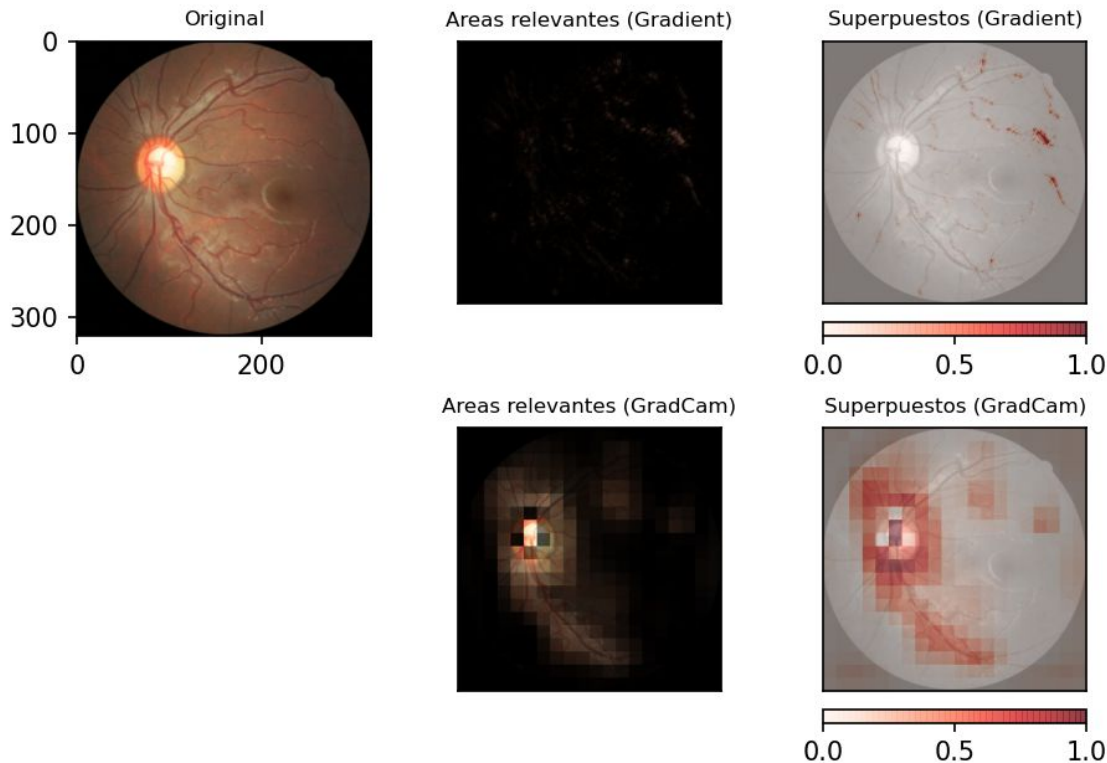
GradCAM

- Calcular $GC = \delta \text{puntuaje}(x) / \delta f_i(x)$
- Promediar GC por features
- Cambiar tamaño a x : GC'
- Dibujar GC' como mapa de calor



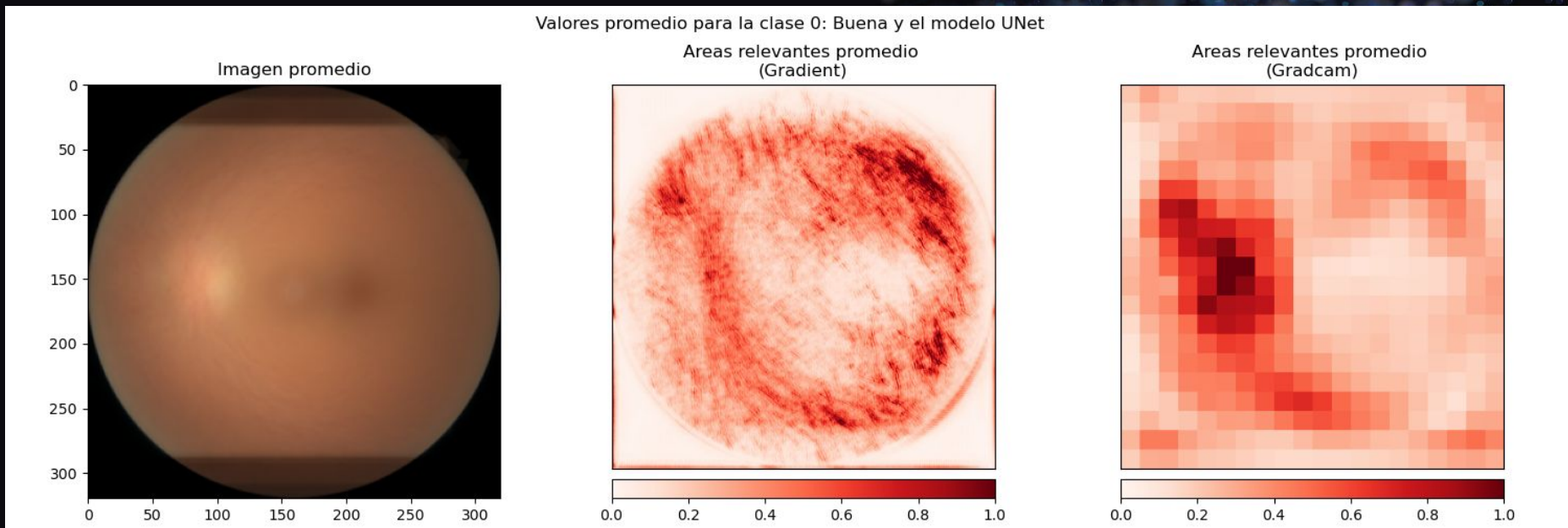
Gradiente y GradCAM (Modelo avanzado, Aceptable)

Zonas relevantes para decidir la calidad de esta imagen.
Calidad de esta imagen: 0: Buena



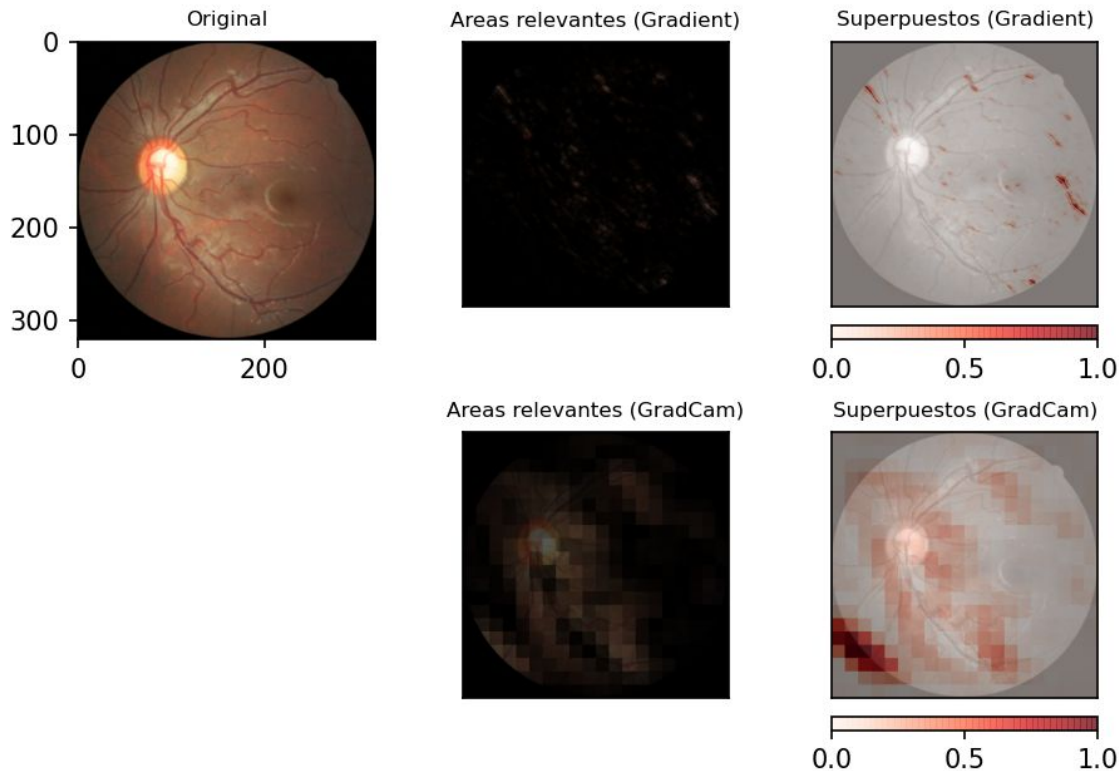
Gradiente y GradCAM (Modelo avanzado, Aceptable)

- Promedio sobre 50 muestras



Gradiente y GradCAM (Modelo simple, Aceptable)

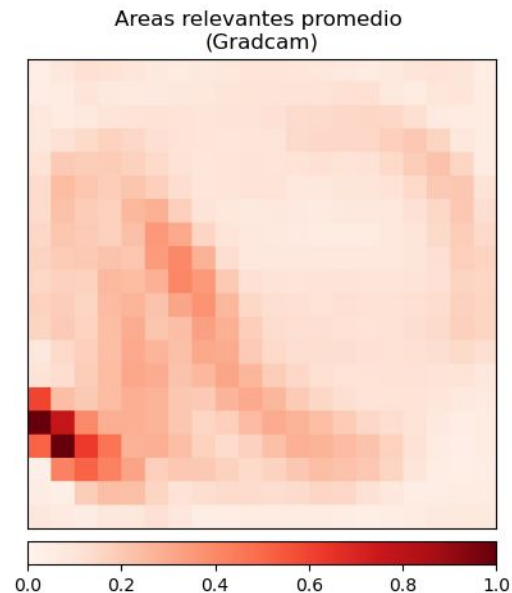
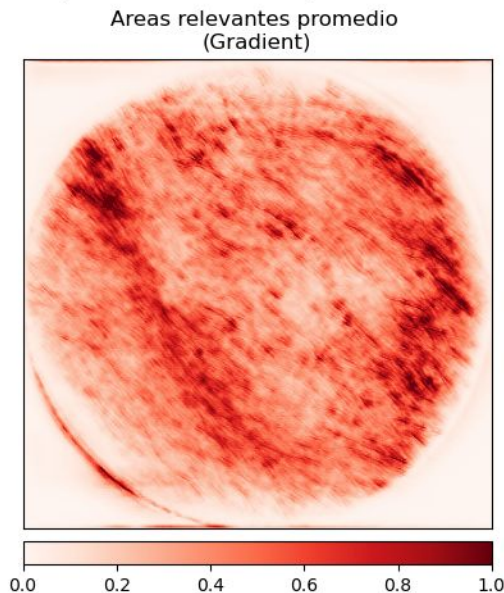
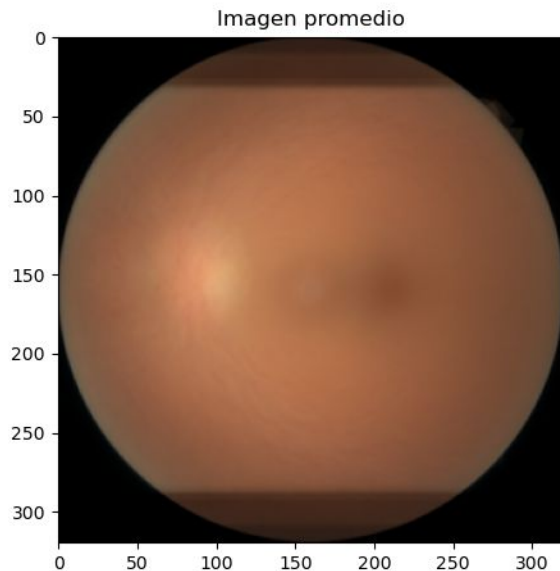
Zonas relevantes para decidir la calidad de esta imagen.
Calidad de esta imagen: 0: Buena



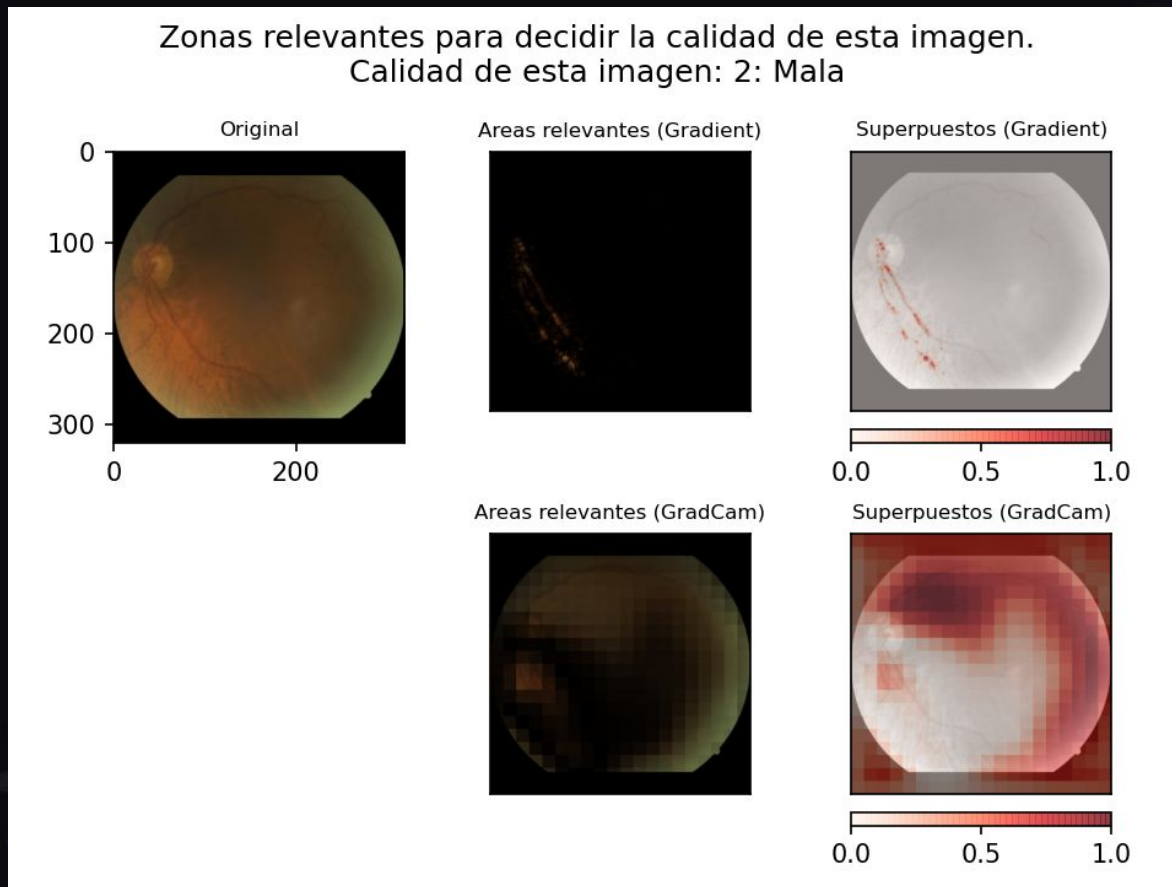
Gradiente y GradCAM (Modelo simple, Aceptable)

- Promedio de 50 muestras

Valores promedio para la clase 0: Buena y el modelo UNetNoDecoder

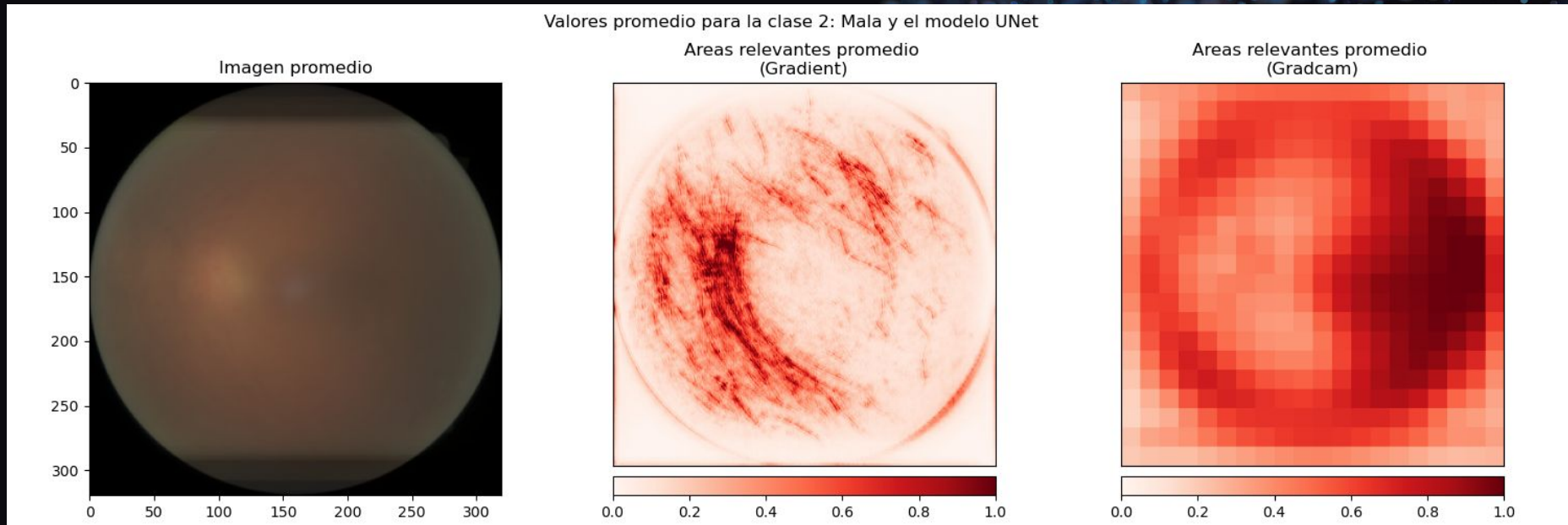


Gradiente y GradCAM (Modelo Avanzado, No Aceptable)



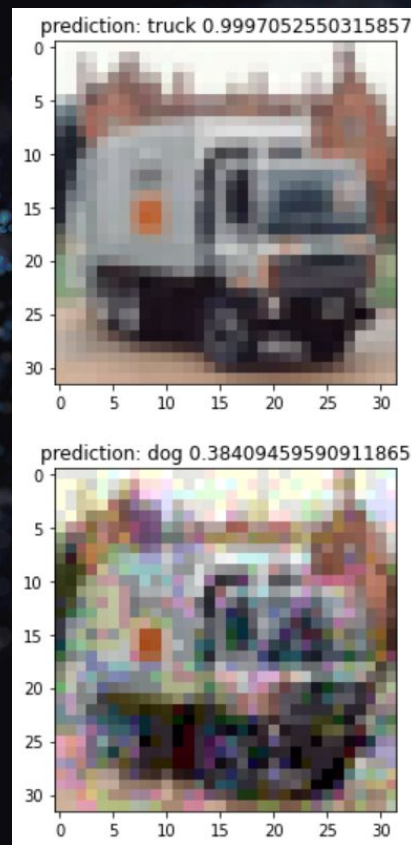
Gradiente y GradCAM (Modelo avanzado, No Aceptable)

- Promedio 50 ejemplos



Ejemplos adversarios

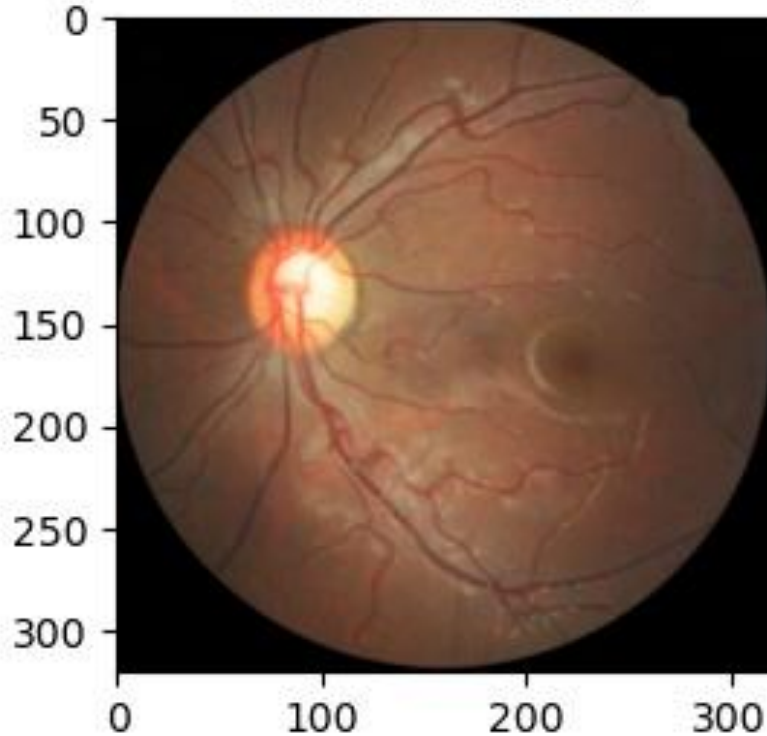
- Elegir ejemplo x de clase C
- Elegir clase objetivo $K \neq C$
- Elegir α (típicamente chico, 0.001)
- Iterar:
 - $x = x + \alpha \delta p_{je_clase}(x,K)/\delta x$
 - hasta $p_{je_clase}(K) \succ p_{je_clase}(C)$



Ejemplos adversarios (Modelo Avanzado)

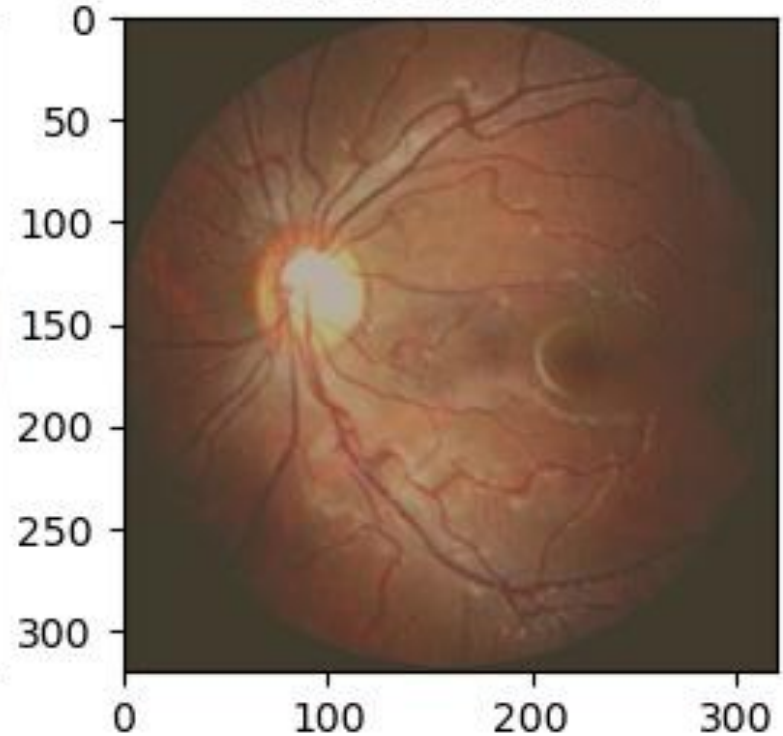
Original class gradable (0)

sco: 0.99,0.01,0.00,,



Target class gradable (0)

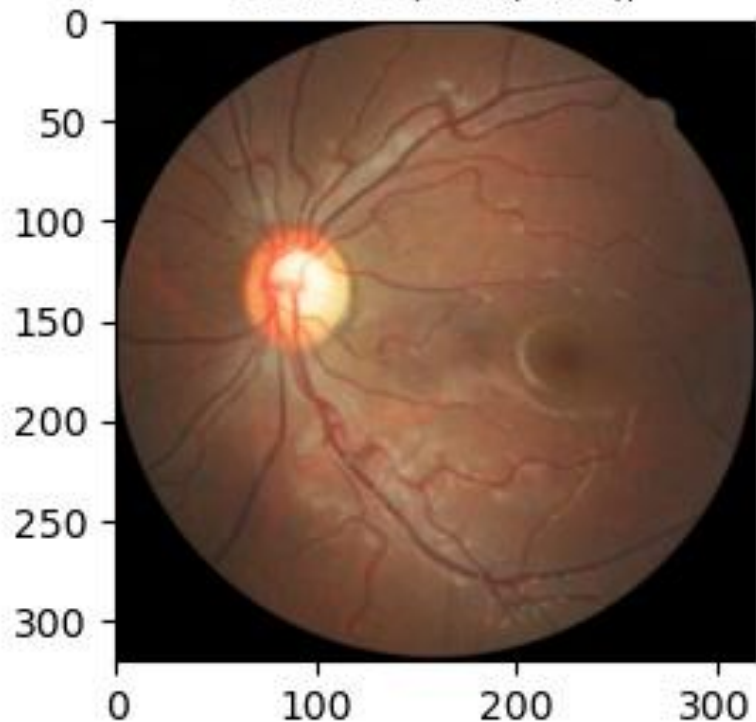
sco: 1.00,0.00,0.00,,



Ejemplos adversarios (Modelo Avanzado)

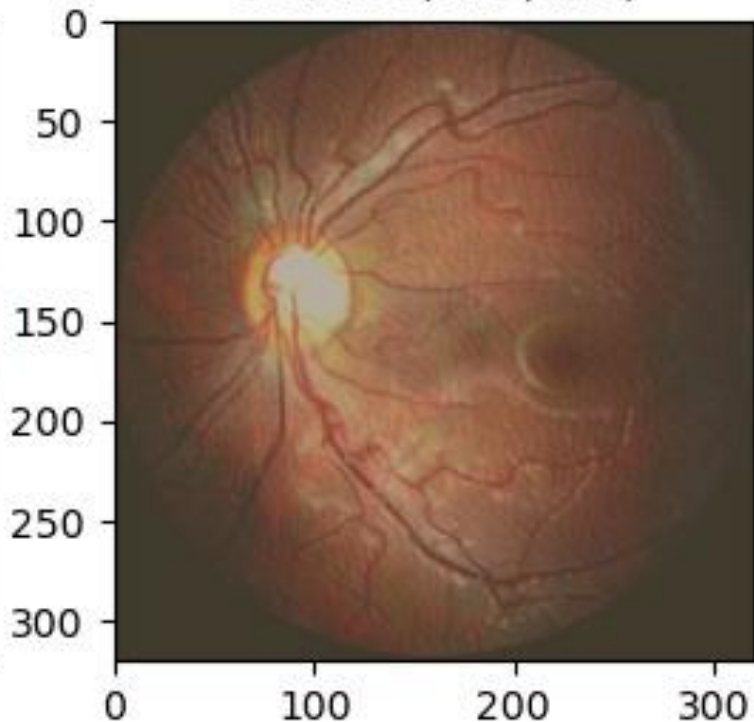
Original class gradable (0)

sco: 0.99,0.01,0.00,,



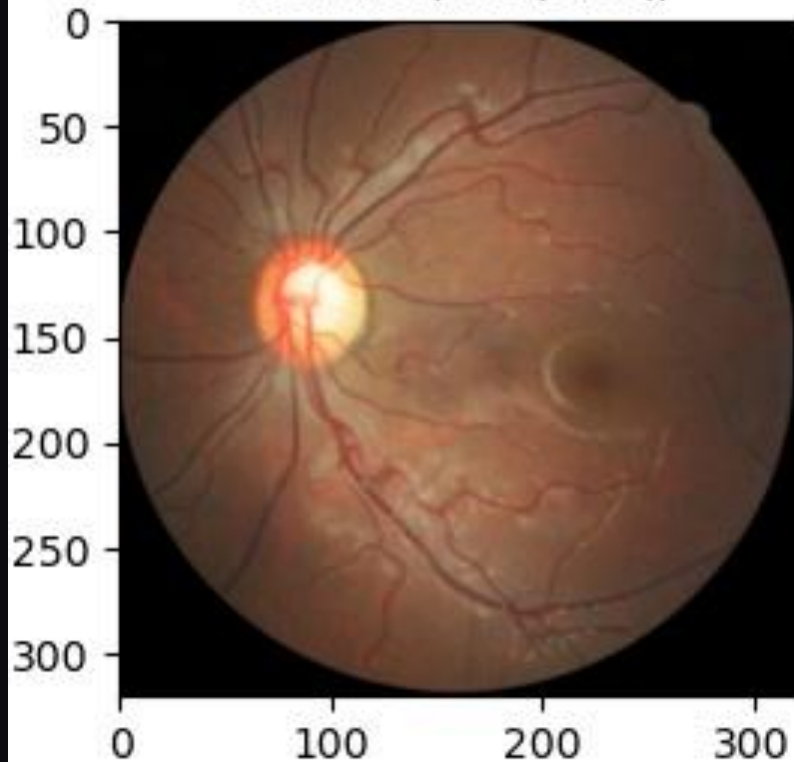
Target class ungradable (2)

sco: 0.00,0.05,0.95,

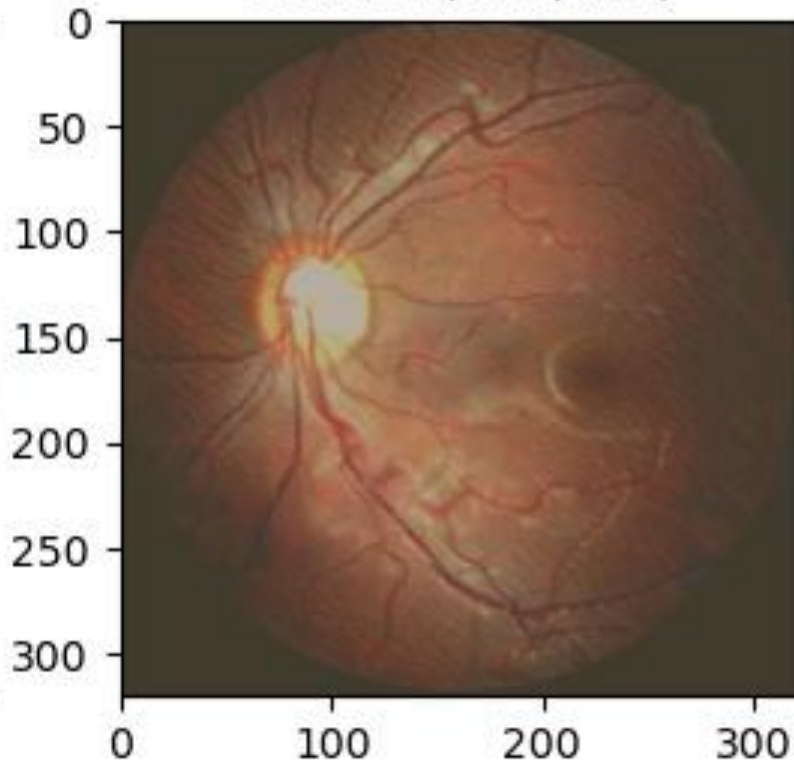


Ejemplos adversarios (Modelo Simple)

Original class gradable (0)
sco: 1.00,0.00,0.00,,



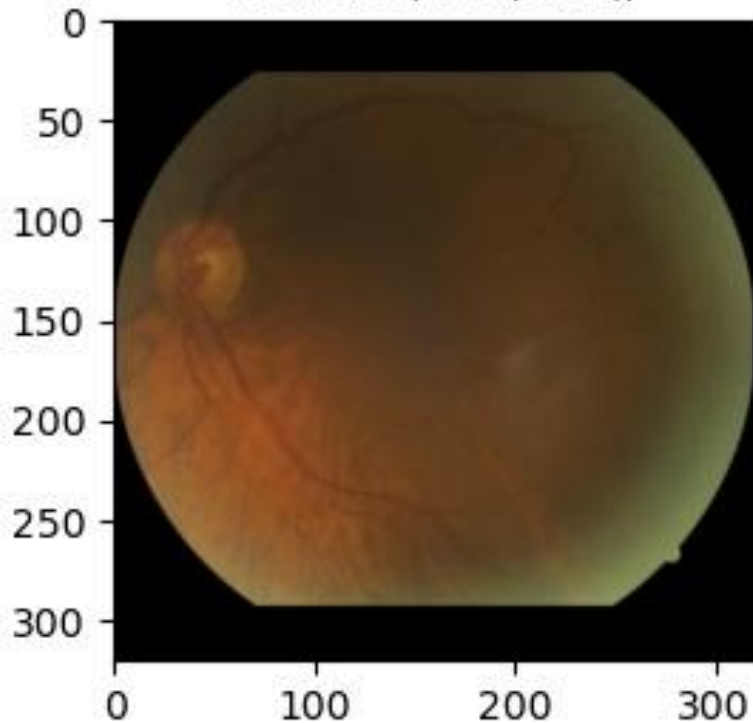
Target class ungradable (2)
sco: 0.02,0.26,0.72,,



Ejemplos adversarios (Modelo Avanzado)

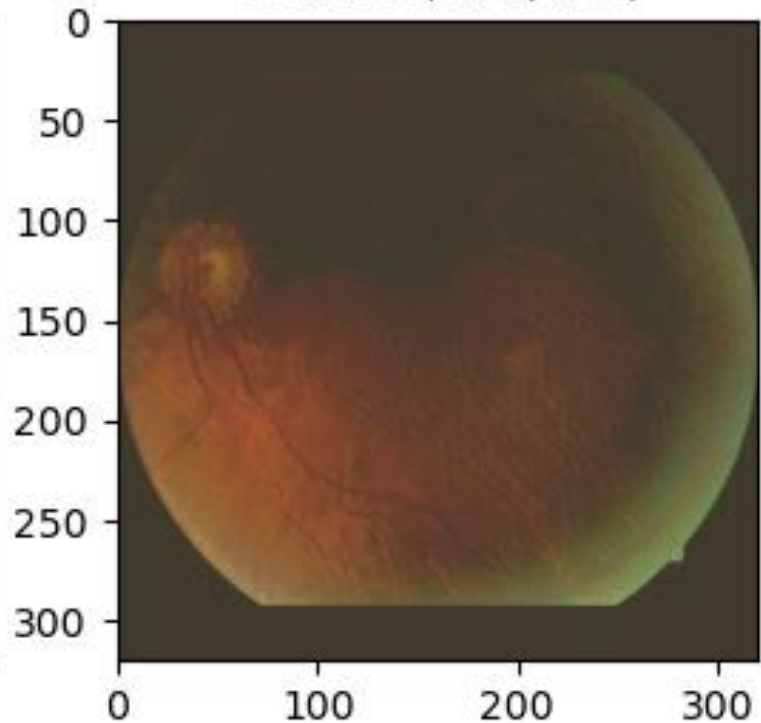
Original class ungradable (2)

sco: 0.00,0.08,0.92,,

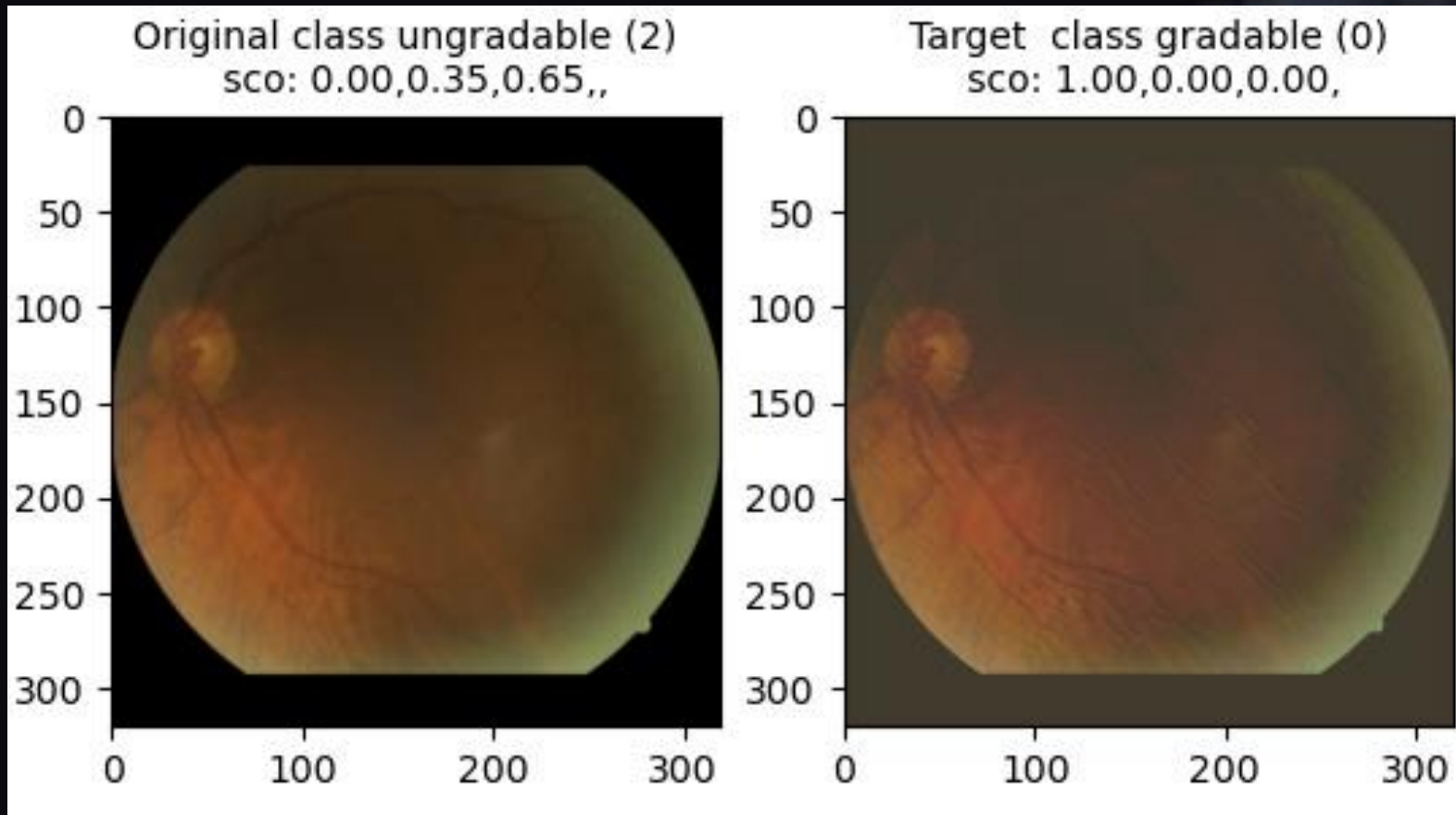


Target class gradable (0)

sco: 0.99,0.01,0.00,,

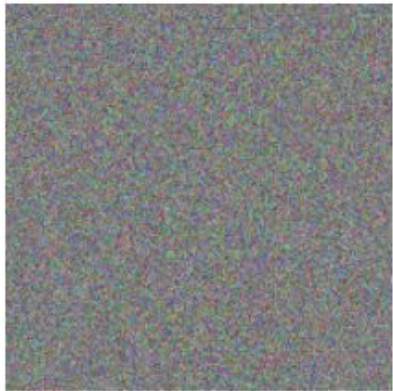


Ejemplos adversarios (Modelo Avanzado)

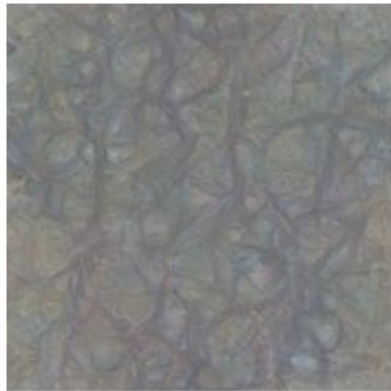


Métodos generativos (colah et al)

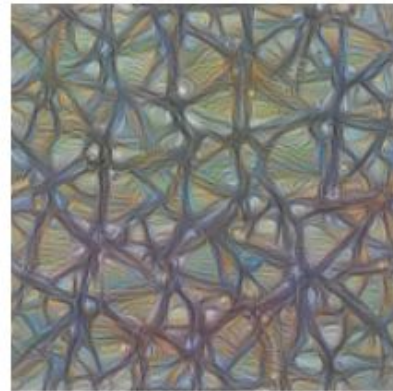
- Encontrar entrada con mejor puntaje: $\text{argmax}_x \text{puntaje}(x)$
[ejemplo de $\text{puntaje}(x)$: softmax de la salida]
- Generar vector de ruido x
 - Iterar hasta converger
 - $x = x + \alpha (\delta \text{puntaje}(x) / \delta(x))$



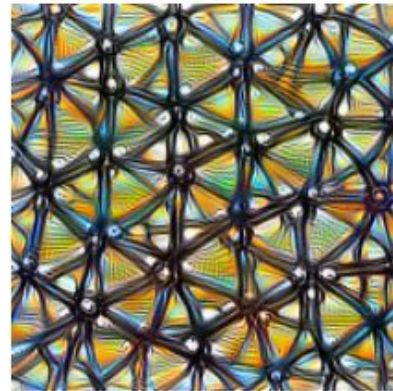
Step 0



Step 4



Step 48



Step 2048

Métodos generativos (colah et al)

- Impacto de la función de puntaje

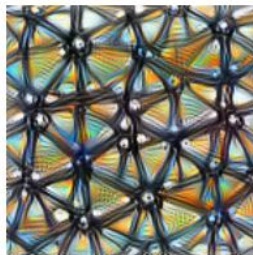
Different **optimization objectives** show what different parts of a network are looking for.

n layer index
x, y spatial position
z channel index
k class index



Neuron

$\text{layer}_n[x, y, z]$



Channel

$\text{layer}_n[:, :, z]$



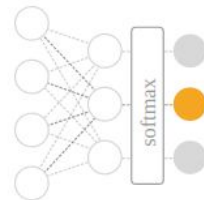
Layer/DeepDream

$\text{layer}_n[:, :, :]^2$



Class Logits

$\text{pre_softmax}[k]$

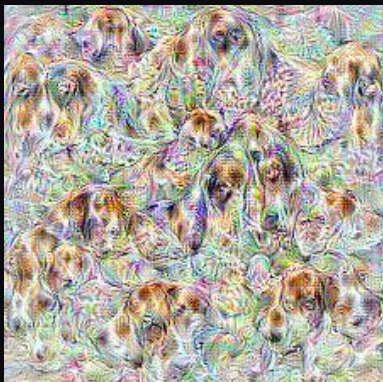


Class Probability

$\text{softmax}[k]$

Métodos generativos con regularización (colah et al)

$$\text{puntaje}'(x) = \text{puntaje}(x) + \text{reg}(x)$$



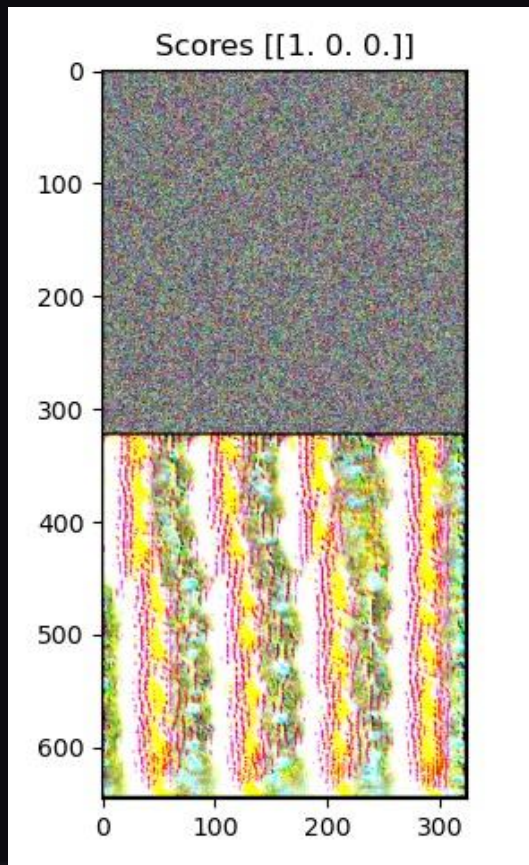
		Weak Regularization avoids misleading correlations, but is less connected to real use.			Strong Regularization gives more realistic examples at risk of misleading correlations.	
		Unregularized	Frequency Penalization	Transformation Robustness	Learned Prior	Dataset Examples
	Erhan, et al., 2009 [3] Introduced core idea. Minimal regularization.	☒	☐	☐	☐	☐
	Szegedy, et al., 2013 [11] Adversarial examples. Visualizes with dataset examples.	☒	☐	☐	☐	☒
	Mahendran & Vedaldi, 2015 [7] Introduces total variation regularizer. Reconstructs input from representation.	☐	☒	☐	☐	☐
	Nguyen, et al., 2015 [14] Explores counterexamples. Introduces image blurring.	☐	☒	☐	☐	☐
	Mordvintsev, et al., 2015 [4] Introduced jitter & multi-scale. Explored GMM priors for classes.	☐	☐	☒	☒	☐
	Øygard, et al., 2015 [15] Introduces gradient blurring. (Also uses jitter.)	☐	☒	☒	☐	☐
	Tyka, et al., 2016 [16] Regularizes with bilateral filters. (Also uses jitter.)	☐	☒	☒	☐	☐
	Mordvintsev, et al., 2016 [17] Normalizes gradient frequencies. (Also uses jitter.)	☐	☒	☒	☐	☐

Optimización de ejemplos con with PyTorch

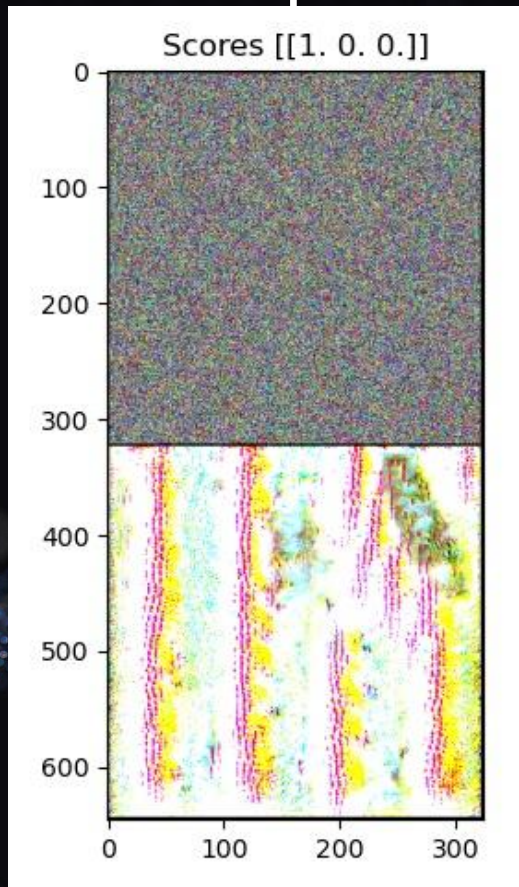
```
def optimize_input(model:nn.Module,class_i:int,x:torch.Tensor,inverse_normalization:Callable,
steps:int, optimizer_generator:Callable,regularizer:Callable):

    blur = GaussianBlur(k=(5,5), s=(0.001,0.001))
    x.requires_grad = True
    optimizer = optimizer_generator([x])
    for i in range(steps):
        class_score = model.forward(x)[: ,class_i]
        regularized_scores = -(class_score+regularizer(x))
        optimizer.zero_grad()
        regularized_scores.backward()
        optimizer.step()
        apply_all(x,blur)
    # apply inverse normalization to put in original feature space
    final_scores = model.forward(x)
    apply_all(x,inverse_normalization)
    return x,final_scores
```

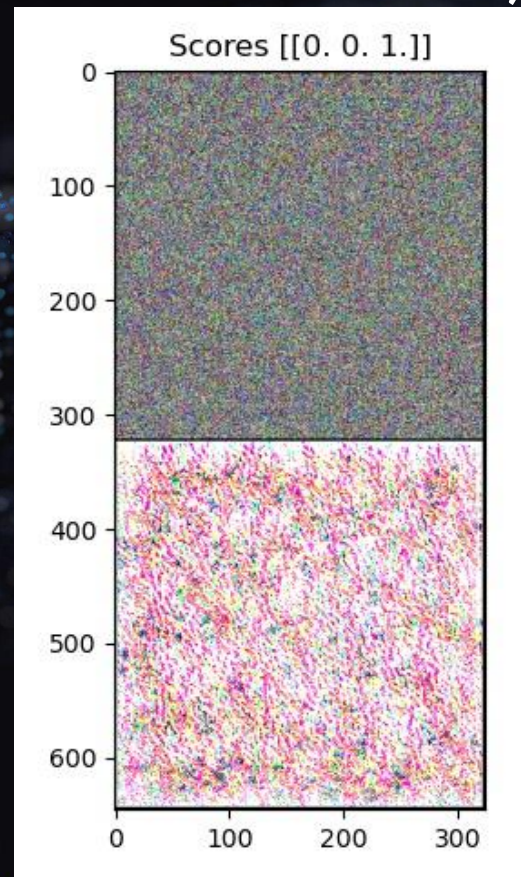
Generación de muestras aceptables (modelo avanzado)



No regularization

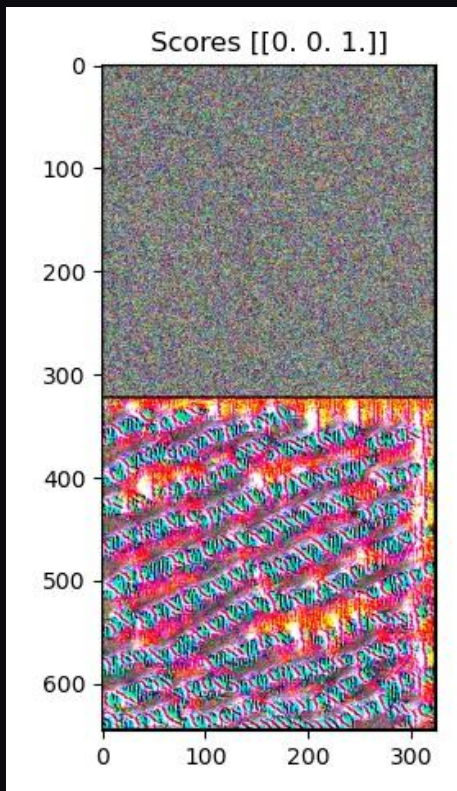


l_2 reg, $\alpha = 0.1$

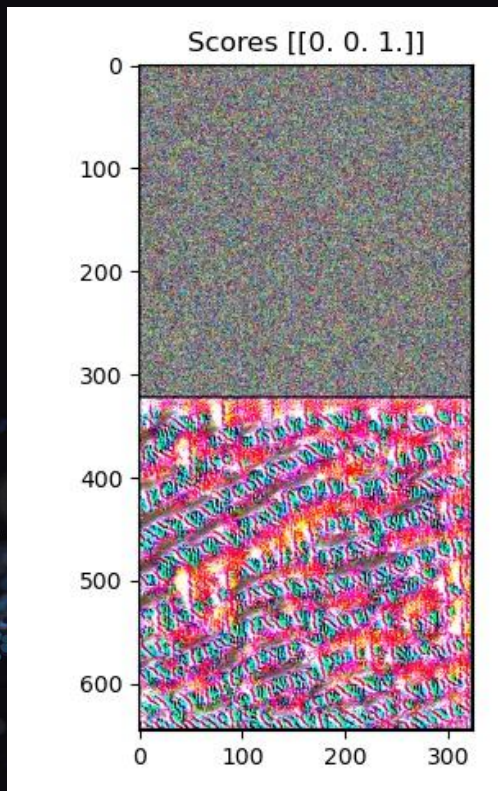


l_2 reg, $\alpha = 1$

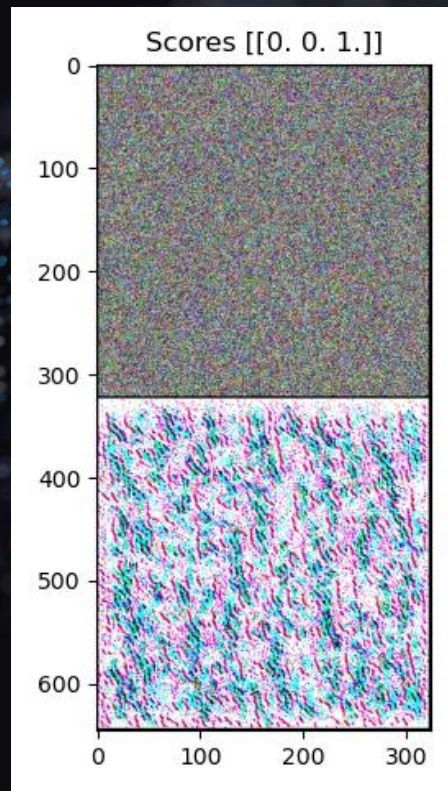
Generación de muestras no aceptables (mod. avanzado)



No regularization

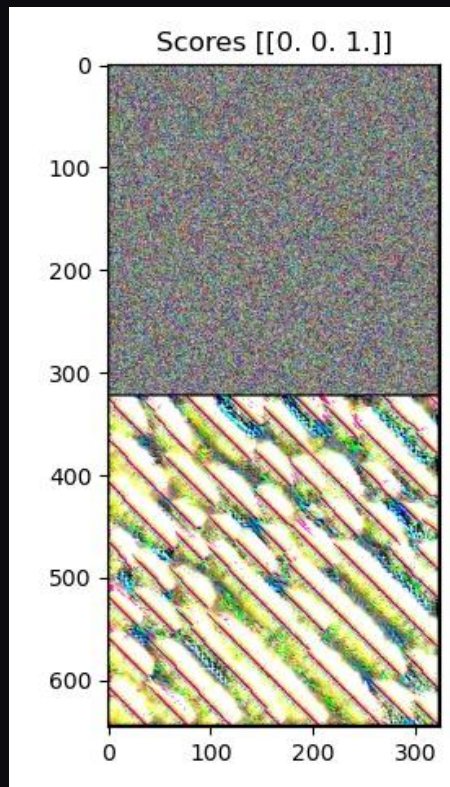


l_2 reg, $\alpha = 0.1$

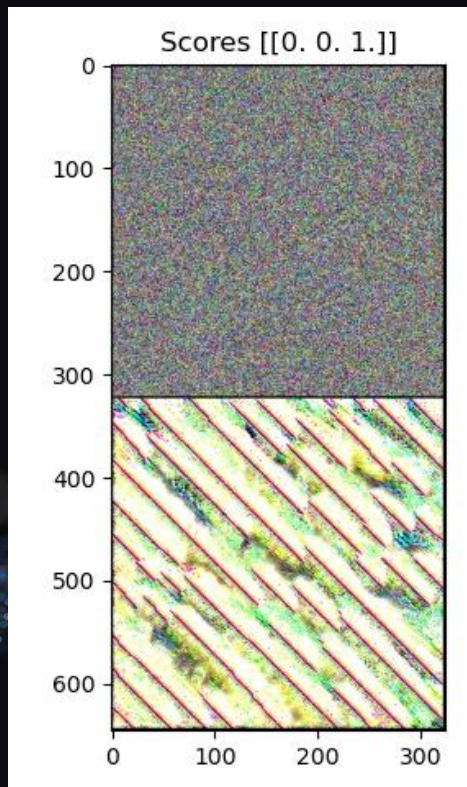


l_2 reg, $\alpha = 1$

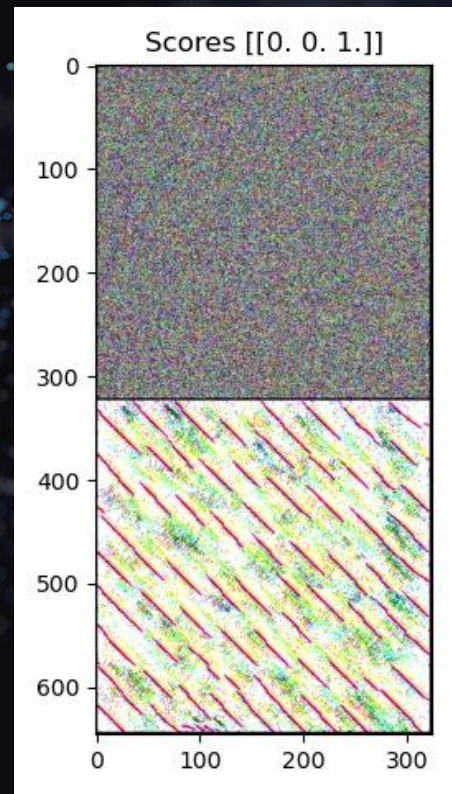
Generación de muestras aceptables (mod. simple)



No regularization



l_2 reg, $\alpha = 0.1$



l_2 reg, $\alpha = 1$

Frameworks para interpretabilidad

- Lucid (Tensorflow)

Visualize a Neuron

```
# Visualizing a neuron is easy!
```

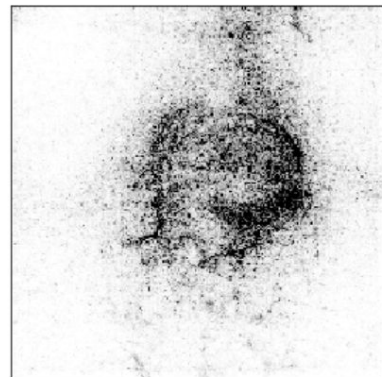
```
_ = render.render_vis(model, "mixed4a_pre_relu:476")
```

512 1150.7921



```
noise_tunnel = NoiseTunnel(integrated_gradients)

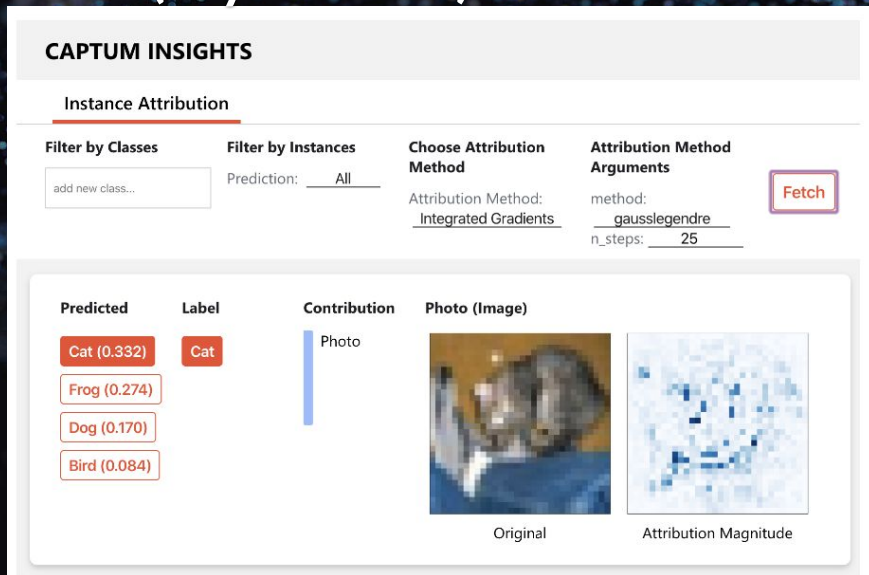
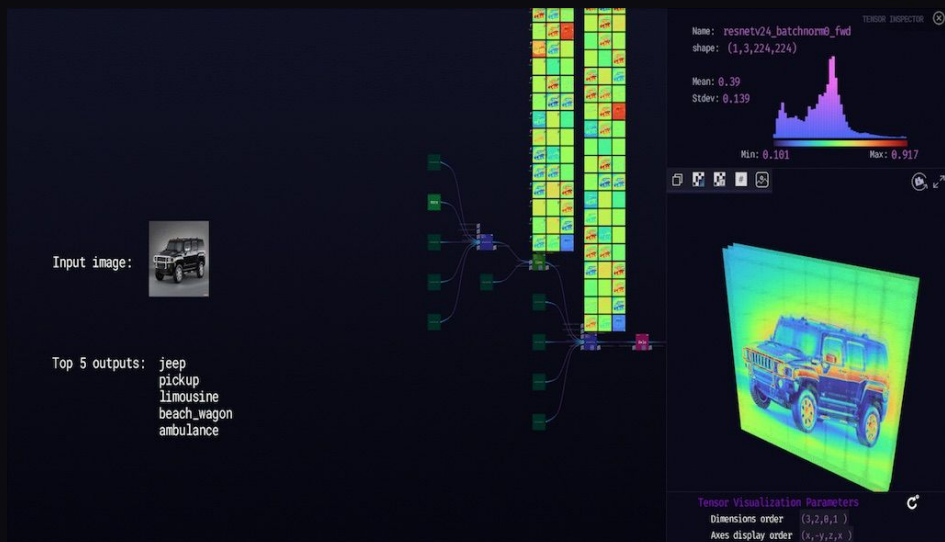
attributions_ig_nt = noise_tunnel.attribute(input, nt_samples=10, nt_
_ = viz.visualize_image_attr_multiple(np.transpose(attributions_ig_nt,
np.transpose(transformed_img.shape[:2], [0, 1]),
["original_image", "heat_map"],
["all", "positive"],
cmap=default_cmap,
show_colorbar=True)
```



- Captum
(PyTorch)

Frameworks para interpretabilidad

- Zetane (ONNX/PyT/TF)
- Captum Insights (PyTorch)



Captum (PyTorch)

```
1 import captum, torch
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 model = ...
6 input = load_image( ... )
7 target_label = ...
8
9 saliency = captum.attr.Saliency(model)
10 grads = saliency.attribute(input, target=target_label)
11 grads = np.transpose(grads.squeeze().cpu().detach().numpy(), (1, 2, 0))
12 plt.imshow(grads)
```

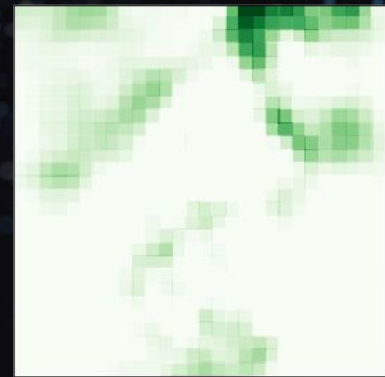
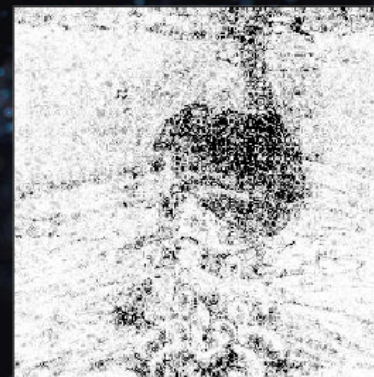
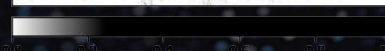
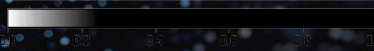
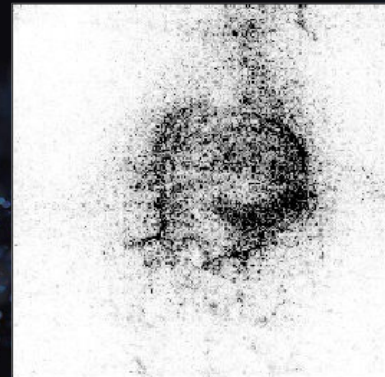
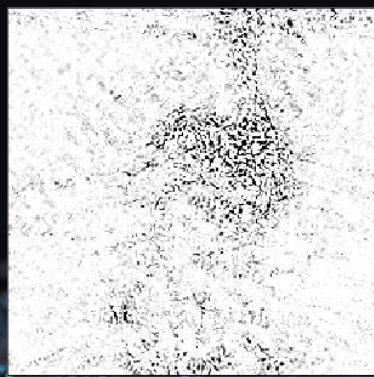


Captum (PyTorch)



Integrated Gradients

IG + Noise



Shap Values

Occlusion Maps